

Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

**Математическое моделирование и реализация планировщика
процессов для гетерогенных процессорных архитектур**

Бакалаврская работа

Научный руководитель:
кандидат физ.-мат. наук, доцент
Корхов В. В.

Рецензент:
эксперт по архитектуре и сетевому стеку,
ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

- Современные процессоры все чаще гетерогенны. Объединяют производительные Р-ядра + энергоэффективные Е-ядра
- Linux EEVDF учитывает типы ядер ограниченно, не балансирует производительность и энергопотребление так, как задумано архитектурой CPU

Цели работы

- 1 Формализовать распределение слотов Р/Е-ядер как динамический аукцион Викри (VCG)
- 2 Реализовать механизм используя sched_ext и оценить на тестах производительности

Каждое ядро рассматривается как **истощаемое благо**, неиспользованный квант времени теряется безвозвратно. Задача-агент характеризуется типом $\theta_i = (v_i, l_i)$, а механизм максимизирует социальное благосостояние.

Аппаратная платформа

- Ядра: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$,
 $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$
- Дискретное время $t = 1, 2, 3, \dots$
- Замедление на E-ядре:
 $\sigma = \eta_P / \eta_E > 1$
- Общий ISA (любая задача может быть выполнена на любом ядре)

Задачи как агенты

- Тип $\theta_i = (v_i, l_i)$ (оценка и длина)
- Полезность $u_i = v_i - p_i$ при полном выполнении
- Бюджет B_i по классу обслуживания:

Реального времени	$B_{RT} \gg 1$
Интерактивный	B_{int}
Пакетный	B_{batch}
Фоновый	$B_{bg} \approx 0$

На E-ядре задача длиной l_i выполняется за $\lceil l_i \cdot \sigma \rceil$ квантов. Ценность v_i начисляется только при полном выполнении (условие полного исполнения по Sano, 2023).

Состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \omega^t)$

- $\mathbf{x}^t$ – оставшиеся кванты на каждом ядре
- $\mathbf{b}^t$ – остатки бюджетов активных задач
- ω^t – внешние факторы (температура, нагрузка)

Действие $a^t = (a_{i,P}^t, a_{i,E}^t)_i$ при ограничениях:

$$a_{i,P}^t + a_{i,E}^t \leq 1, \quad \forall i$$

$$\sum_i a_{i,P}^t \leq K_P^t, \quad \sum_i a_{i,E}^t \leq K_E^t$$

Вознаграждение

Задача i (покупатель):

$$r_i(s, a) = v_i \cdot \mathbb{1}[i \text{ завершилась в } t]$$

Планировщик (продавец):

$$r_0 = \mu_P \sum_{k \in \mathcal{P}} \mathbb{1}[x_k \geq 1] + \mu_E \sum_{k \in \mathcal{E}} \mathbb{1}[x_k \geq 1]$$

μ_κ – вознаграждение за активный квант ядра типа κ ; $\mu_P > \mu_E$ (Р-ядро производительнее)

Социальное благосостояние

$$W(\pi) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} \left[\sum_{t=1}^T R(s^t, a^t) \right]$$

Эффективная ценность задачи i на ядре типа $\kappa \in \{P, E\}$

$$\phi_P(\theta_i) = v_i - c_P \cdot l_i, \quad \phi_E(\theta_i) = v_i - c_E \cdot \lceil l_i \sigma \rceil$$

Оптимальное распределение (максимизация W)

$$a^t = \arg \max_a \sum_i [a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)] + \delta \mathbb{E}[W(s^{t+1})]$$

$\delta \in [0, 1)$ – дисконт будущего благосостояния

Платёж VCG задачи i

$$p_i^t = W_{-i}(s^t) - (W(s^t) - \phi_\kappa(\theta_i))$$

$W(s^t)$, $W_{-i}(s^t)$ – оптимальное социальное благосостояние с i и без неё. Задача i платит ровно то, что теряют остальные агенты из-за её присутствия (pivot rule).

Свойства механизма

- Стимульная совместимость (в пределах класса, см. Extra)
- Индивидуальная рациональность $u_i \geq 0$
- Бюджетное ограничение $p_i \leq B_i^t$

sched_ext (scheduler extensions) — инфраструктура ядра Linux, позволяющая реализовывать политику планирования как eBPF-программу, подключаемую к интерфейсу расширяемых планировщиков без пересборки ядра

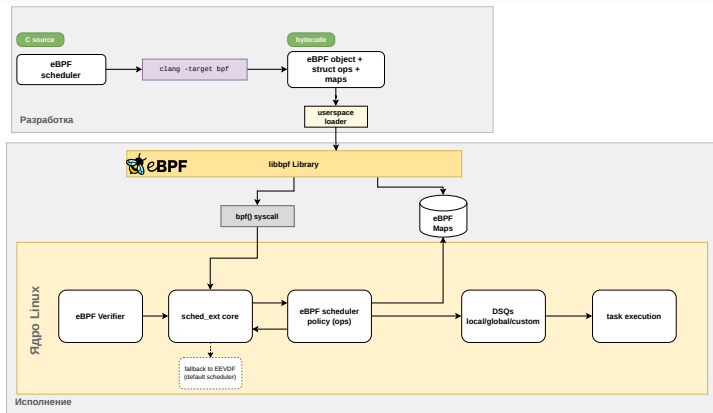


Рис. 1. Механизм работы sched_ext

DSQ (dispatch queue)

- SCX_DSQ_LOCAL
- SCX_DSQ_GLOBAL
- пользовательские DSQ

Цикл планирования

- select_cpu
- enqueue
- dispatch
- running / stopping

Сигнал маршрутизации – отношение бюджета $\rho_i = B_i / B_i^{\max}$ (высокое $\rho \Leftrightarrow$ много накопленного бюджета $\Leftrightarrow$ недавний простой)

DSQ_P

$$\rho_i \geq \rho^* (50\%)$$

Задача короткими всплесками потребляет CPU и большую часть времени ждёт IO. Выигрывает от пиковой частоты P-ядра

DSQ_E

$$\rho_i < \rho^*$$

Задача устойчиво расходует бюджет, близка к CPU-bound переносима на E-ядро

DSQ_Starved

$$B_i < B_i^{\max} / 10$$

Бюджет почти исчерпан штраф к виртуальному дедлайну $v_d += \max(0, -\phi_\kappa) / D$ демоция, не блокирует остальных

Прирост бюджета за время простоя $B_i += \Delta t_{\text{idle}} \cdot v_i / D_{\text{rep}}$ играет роль, аналогичную пороговому правилу Майерсона в одномерных аукционах

Система

Ядро Linux 7.0
CPU: Intel Core
Ultra 7 265K
8P + 12E

Сравнение

Linux EEVDF
vs
scx_EEVDF
vs
scx_LAVD
vs
scx_auction
(VCG)

Бенчмарки

- **hackbench** – 10 групп по 40 задач, обмениваются короткими сообщениями через сокеты. Генерирует большое количество переключений контекста (стресс тест для планировщиков)
- **sysbench** – нагрузка на CPU, память и дисковый ввод-вывод с параметрами по умолчанию. Имитирует смешанную OLTP-нагрузку баз данных
- **sched_latency** – собственный тест, работает параллельно с нагрузкой. Регистрирует, сколько задача ждёт между моментом готовности и началом исполнения

Метрики

hackbench

Время (s), ↓

sysbench

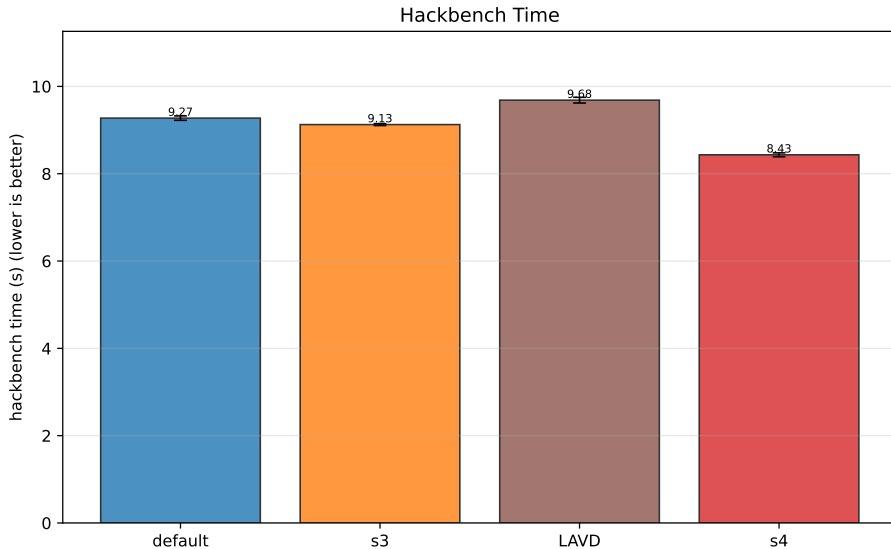
events/s, ↑

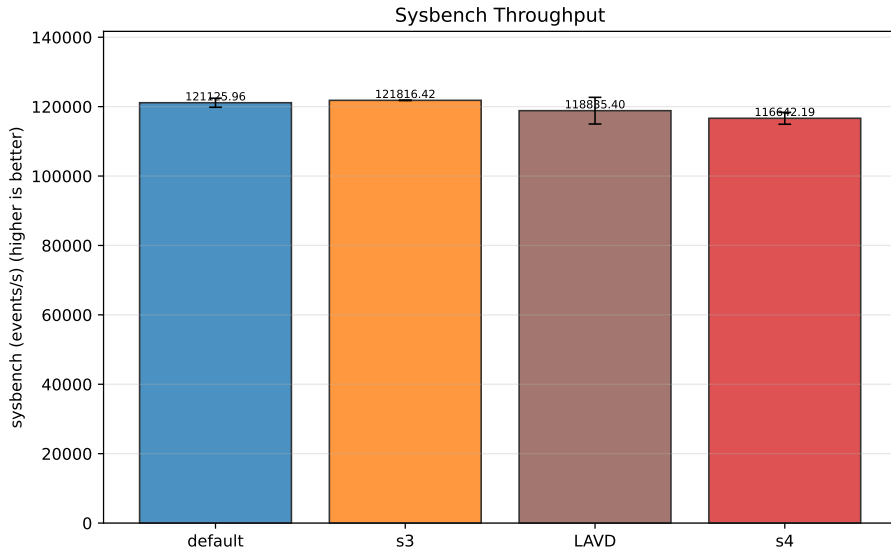
sched_latency

p99 (μ s) по
типам

RAPL

package (W)





Результаты. Итоговая таблица

Метрика	Linux EEVDF	scx_EEVDF	scx_LAVD	scx_auction
Hackbench (s) ↓	9.27	9.13	9.68	8.43
Sysbench (ev/s) ↑	121 126	121 816	118 835	116 642
Sched delay avg (μ s) ↓	2172	4620	361	3629
Sched delay p99 (μ s) ↓	9605	15 246	3014	38 025
Preemption avg (μ s) ↓	8878	18	9709	1055
Context sw (K/s) ↓	187	46	191	62
Sleep dur. avg (ms) ↓	130	149	113	26
Power (W) ↓	175.7	187.8	167.4	186.5

- Распределение слотов P/E-ядер формализовано как динамический аукцион VCG: состояние, действие, эффективная ценность ϕ_k , платёж по pivot-правилу. Обоснованы стимульная совместимость в пределах класса, индивидуальная рациональность и бюджетное ограничение
- Механизм реализован как BPF-планировщик `scx_auction` поверх `sched_ext` (DSQ_P / DSQ_E / DSQ_Starved, маршрутизация по ρ_i , прирост бюджета за idle как аналог порогового правила Майерсона)
- На Intel Core Ultra 7 265K (8P+12E) получено лучшее время `hackbench` (8.43s, -9% к Linux EEVDF) и наименьшая средняя длительность сна (26ms). Sysbench и энергопотребление сопоставимы, p99 sched-delay проигрывает `scx_LAVD` – очередное направление работы

Спасибо за внимание!

- [1] Leon G., Etesami O. Mechanism Design for Scheduling with Heterogeneous Workers. NeurIPS, 2022
- [2] Sano T. Dynamic Mechanism Design for Multi-Period Contracting. Working paper, 2023
- [3] Myerson R. B. Optimal Auction Design. Mathematics of Operations Research, 6(1), 1981, pp. 58-73
- [4] Stoica I., Abdel-Wahab H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Old Dominion University, 1996
- [5] Humphries J.T., Natu N., Chaugule A. et al. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling. ACM SOSR, 2021
- [6] Van Craeynest K., Jaleel A., Eeckhout L. et al. Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE). ISCA, 2012, pp. 213-224
- [7] Galin M., Hedblom A. Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads. Linköping University, 2025
- [8] The extensible scheduler class (LWN)
- [9] An EEVDF CPU scheduler for Linux (LWN)

Стимульная совместимость (Sano, Теорема 1)

Механизм стимульно совместим тогда и только тогда, когда для каждой задачи i :

- 1 $\alpha_i(v_i, l_i)$ не убывает по v_i при фиксированном l_i
- 2 $\int_0^{v_i} \alpha_i(\nu, l_i) d\nu$ не возрастает по l_i при фиксированном v_i
- 3 Существует $\underline{\Pi}_i$, не зависящее от l_i :

$$\Pi_i(v_i, l_i) = \underline{\Pi}_i + \int_0^{v_i} \alpha_i(\nu, l_i) d\nu$$

Задачам невыгодно завышать оценку v_i или занижать длину l_i —правдивая ставка является доминирующей стратегией

Онлайн-обучение: IHMDP-VCG (Leon & Etesami)

При неизвестных распределениях типов задач и ядре MDP планировщик обучается онлайн. Эпизодическая структура:

- **Фаза перемешивания** (длина $d^{[k]}$): вывод цепи Маркова на стационарное распределение
- **Стационарная фаза** (длина $l^{[k]}$): сбор платежей $\hat{p}^{[k]}$, обновление оценок

Гарантии при $T \rightarrow \infty$ (с вер. $\geq 1 - O(\zeta)$):

Регрет благосостояния	$\tilde{O}(\varepsilon T n + T^{2/3} n)$
Регрет продавца	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$
Регрет покупателей	$\tilde{O}(\varepsilon T n^2 + T^{2/3} n^2)$

Асимптотически:
 стимульная индивидуальная одновременно
 эффективность, совместимость и рациональность —

Пример: $K_P = K_E = 1$, одно свободное ядро типа κ

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa$$

где $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$

Платёж победителя:

$$p_{i^*} = \phi_\kappa^{-1} \left(\phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa \mid l_{i^*} \right)$$

где j — задача на втором месте.

$\phi_P(\theta_i) > \phi_E(\theta_i)$ выполняется не всегда: при малом v_i и большом l_i Е-ядро может дать большую эффективную ценность ($c_E \cdot \lceil l_i \sigma \rceil < c_P \cdot l_i$ при $\sigma < \gamma$). Оптимальная стратегия нетривиальна.

IC при жёстком бюджетном ограничении

Проблема. В классическом VCG $p_i^{\text{VCG}} > B_i$ создаёт стимул к искажению ставки, чтобы попасть в допустимое множество — IC ломается (Dobzinski, Lavi, Nisan 2008).

Наш случай. Бюджет B_i — функция класса обслуживания $\kappa_i \in \{\text{RT}, \text{int}, \text{batch}, \text{bg}\}$, фиксированного приоритетом процесса (policy/nice) до поступления в механизм. Класс — public type, не объект стратегии.

Утверждение. При фиксированном классе κ и соответствующем бюджете $B(\kappa)$ механизм IC на подтипе (v_i, l_i) : условия Sano (Т. 1) выполняются для α_i^κ , платёж $p_i \leq B(\kappa)$ не зависит от сообщения агента $\Rightarrow$ правдивое (v_i, l_i) — доминирующая стратегия внутри класса.

Между классами — approximate IC; задача с $p_i^{\text{VCG}} > B_i$ отсекается аллокацией (§8.3 модели), не перепрайсингом.